

VIET NAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE



NHẬP MÔN DỮ LIỆU LỚN

Lecturers

Dr. Le Ngoc Thanh | lnthanh@fit.hcmus.edu.vn

Dr. Nguyen Ngoc Thao | nnthao@fit.hcmus.edu.vn

Lab Instructors

MSc. Huynh Lam Hai Dang | hlhdang@fit.hcmus.edu.vn

Tran Huy Ban | huyban.han@gmail.com

Lab 04: Spark Streaming

1. Statements

This laboratory assignment focuses on incremental Code Property Graph (CPG) construction combined with a real-time streaming ingestion pipeline. Each student team must select one publicly available Python repository drawn from one or more designated GitHub organizations, construct an incremental CPG from its Python source files, and build a streaming pipeline that persists the resulting graph and code metadata into two separate database systems.

Repository selection is managed through the course Moodle page via a Choice activity. Each available repository will have a fixed maximum number of teams that may work on it. Once that quota is filled, the repository becomes unavailable for further selection. Teams must complete their selection before the deadline specified on Moodle. No two teams working on the same repository are permitted and late selection may result in reduced choice of repositories.

After selection is confirmed, each team clones the repository and begins implementing the pipeline as described in the sections below.

Disclaimer:

This exercise is for educational and technical demonstration purposes only. It is hereby declared that no encouragement, endorsement, or recommendation for engaging in security analysis, vulnerability identification, or cybersecurity threat intelligence is provided.

1.1. Objectives

- Understand the structure of a Code Property Graph. The relevant elements are abstract syntax tree (AST) nodes, control flow graph (CFG) edges, data flow graph (DFG) edges, and call edges. Together these describe the syntactic and semantic structure of source code for static program analysis.
- Implement an incremental Python parser that processes source files one at a time and emits structured events to an Apache Kafka topic cluster.
- Design and implement the Kafka topic layout used by the pipeline to carry graph node events, graph edge events, and source metadata events.
- Ingest the graph topology into Neo4j using the Neo4j Kafka Connector Sink without an intermediate Spark layer.

- Ingest source code metadata into MongoDB using Apache Spark Structured Streaming with the MongoDB Spark Connector.
- Demonstrate idempotent pipeline replay by reprocessing at least one modified Python file and verifying that Neo4j and MongoDB reflect the updated state without duplication.

1.2. Repository Cloning and File Discovery

Use git to shallow-clone your assigned repository and reduce download size. The clone URL will follow the standard pattern of `https://github.com/<assigned-org>/<assigned-repo>.git`. After cloning, enumerate all .py source files within the repository tree. Excluding test files, setup files, and auto-generated files is optional but recommended. Record the total number of discovered Python files in your lab report.

1.3. Incremental CPG Parser Service

Build a Python service called the Parser Service that processes Python source files one file at a time rather than the entire repository in a single batch. The service uses a Python-side CPG and AST parsing library of the team's choice, drawn from Joern, tree-sitter, or the standard library ast module, to extract the AST nodes, CFG edges, DFG edges, and call edges that constitute the CPG. Each category is then emitted as a structured event message to an Apache Kafka topic. The service should operate within bounded memory and must assign stable identifiers to every emitted element so that reprocessing the same content does not create duplicates downstream.

1.4. Kafka Topic Design

Design the Kafka topic layout that the pipeline uses to carry the four categories of events emitted by the Parser Service. Using Apache Kafka as the message broker and topic management surface, the layout must include separate topics for node events, edge events, source metadata events, and parser error events. Each message must carry a schema version field for forward compatibility and an event time timestamp.

1.5. Kafka Topic Design

Wire the Neo4j Kafka Connector Sink to the topics carrying node and edge events so that the graph topology is written into Neo4j directly from Kafka without an intermediate Spark layer. The ingestion logic must be idempotent so that reprocessing the same node or edge does not create duplicates.

1.6. Kafka Topic Design

Build an Apache Spark Structured Streaming job that consumes the metadata events from Kafka and writes them through the MongoDB Spark Connector into a MongoDB collection. The job must use a checkpoint location so that it can resume from the last processed offset on restart.

1.7. Kafka Topic Design

Modify one Python source file in your cloned repository and reprocess that single file through the Parser Service. Verify that the node and edge counts in Neo4j reflect the update without creating duplicate nodes, that the MongoDB collection contains the updated metadata document for that file, and that the Apache Spark Structured Streaming checkpoint correctly skips already-processed offsets for all unchanged files.

2. Submission Guideline

Each team must submit a single URL pointing to a published Jupyter Book hosted on GitHub Pages. The Jupyter Book must be served from a public GitHub repository owned by the team and must be organized as a structured narrative that walks through each task in sequence.

Each chapter or section of the Jupyter Book corresponds to one task in the lab. The content of each chapter must include written explanation of the approach the team chose and the reasoning behind it. It must also include executed notebook cells showing real intermediate outputs such as parsed node counts, Kafka message samples, and database query results. The chapter must further include screenshots or embedded figures of database UI views. Each chapter ends with a brief reflection on what worked, what failed, and how the team resolved any issues encountered.

The GitHub repository that hosts the Jupyter Book must also contain all source code the team wrote, organized in a logical folder structure, and committed with meaningful commit messages that reflect the team's incremental progress over the duration of the lab.

The submission on Moodle is therefore exactly one text entry, which is the root URL of the published Jupyter Book site. No zip files, no PDF exports, and no Word documents are accepted.

Grading Criteria

The grading criteria are summarized in the below table.

Requirements	Points
Task 1. Repository Cloning and File Discovery	1
Task 2. Incremental CPG Parser Service	1.5
Task 3. Kafka Topic Design	1.5
Task 4. Graph Topology Ingestion into Neo4j	2
Task 5. Source Metadata Ingestion into MongoDB	2
Task 6. Idempotent Replay Verification	1
Architecture Diagram	1
TOTAL	10

Also note that:

- Ensure your code is well-documented with clear comments.
- Include all necessary files, logs, and screenshots to verify successful execution.
- Each task can be accomplished under complex environments and different programming languages, remember to provide instructions for running each task if this is the case.

Happy Coding and Best of Luck!

The Instructor./.